UNIVERSIDAD PONTIFICIA COMILLAS · ICAI
Grado en Ingeniería Matemática e Inteligencia Artificial

Fundamentos de Inteligencia Artificial · Proyecto final

Agentes inteligentes en un escenario basado en *Apocalypse Now*

Agente lógico, agente bayesiano y proceso de decisión de Markov

Resumen

Este proyecto modela la misión del Capitán Willard, inspirada en la película *Apocalypse Now*, mediante tres agentes de inteligencia artificial. En la primera parte, un **agente lógico** con base de conocimiento guía al jugador por un palacio de 6×6 casillas con precipicios, un soldado y una salida ocultos. En la segunda, un **agente bayesiano** mantiene distribuciones de probabilidad sobre la posición de tres trampas, el soldado y la salida, las actualiza con la regla de Bayes y recomienda los movimientos cuyo riesgo es inferior a un umbral. En la tercera, el cruce de un río con corriente se formula como un **proceso de decisión de Markov** y se resuelve con *value iteration*. Los dos primeros agentes asisten a un jugador humano, que es quien decide cada acción; el tercero calcula una política óptima y la simula de forma autónoma.

Autor Jaime Tuesta de Simón
Curso 2.º del Grado en Ingeniería Matemática e Inteligencia Artificial
Fecha Enero de 2026

Índice

1 Introducción **2**

2 Entorno del palacio **2**

2.1 Estructura . . . 2

2.2 Acciones . . . 2

2.3 Perceptos . . . 3

3 Parte 1: agente lógico **3**

3.1 Base de conocimiento . . . 3

3.2 Inferencia . . . 4

3.3 Flujo de ejecución e interfaz . . . 4

3.4 Resultados . . . 5

4 Parte 2: agente bayesiano **5**

4.1 Creencias iniciales . . . 5

4.2 Actualización con la regla de Bayes . . . 5

4.3 Riesgo de cada casilla . . . 6

4.4 Mapas de calor . . . 6

4.5 Flujo de ejecución . . . 7

4.6 Resultados . . . 7

5 Parte 3: proceso de decisión de Markov **8**

5.1 El río . . . 8

5.2 Componentes del MDP . . . 8

5.3 Value iteration . . . 8

5.4 Simulación . . . 9

5.5 Resultados . . . 9

6 Conclusiones **10**

1 Introducción

El proyecto representa la misión del Capitán Willard para localizar al Coronel Kurtz y escapar con él. Se divide en tres programas independientes, que se resumen en la Tabla 1.

Tabla 1. Programas del proyecto.

Programa	Técnica	Problema
`kurtz.py`	Agente lógico con base de conocimiento	Palacio con tres precipicios, un soldado dormido, Kurtz y la salida.
`palacio.py`	Agente bayesiano	Palacio con tres trampas (fuego, pinchos y dardos), un soldado, Kurtz y la salida.
`river_mdp.py`	MDP y *value iteration*	Cruce de un río con corriente e islas hasta la salida.

En los dos escenarios del palacio se sigue la primera de las opciones del enunciado: un jugador controla a Willard, y el agente le presenta en cada turno la información que ha inferido, le indica qué casillas son seguras y hacia dónde debe dirigirse. En el escenario del río, el agente calcula una política óptima y la ejecuta por sí mismo.

2 Entorno del palacio

2.1 Estructura

El palacio se modela como una cuadrícula de 6×6 casillas generada aleatoriamente en cada ejecución. Willard empieza siempre en la casilla $(0, 0)$ y el resto de elementos se colocan al azar (Tabla 2). Todas las casillas están ocultas al principio: el jugador solo ve las casillas que ha visitado.

Tabla 2. Elementos del palacio en cada parte.

	Parte 1: agente lógico	Parte 2: agente bayesiano
Peligros	3 precipicios (P), letales	3 trampas: fuego (F), pinchos (P) y dardos (D), letales; pueden coincidir en una casilla
Soldado	S: mata a Willard; puede morir con la granada	M: igual que en la parte 1
Kurtz	K: hay que rescatarlo antes de salir	K: igual que en la parte 1
Salida	E	S
Restricciones	Cada elemento ocupa una casilla vacía	Ningún elemento en $(0, 0)$; el soldado, Kurtz y la salida nunca están en una casilla con trampa

2.2 Acciones

El jugador dispone de tres tipos de acción:

- **Moverse** (`W`, `A`, `S`, `D`), con `move_captain()`. Willard muere si entra en una casilla con un peligro o con el soldado vivo, y rescata a Kurtz automáticamente al entrar en su casilla.
- **Lanzar la granada** (`G`) a una casilla adyacente, con `throw_grenade()`. Si acierta, el

soldado muere; solo hay una granada.

- **Salir** (`E`), con `exit_palace()`. El juego termina con éxito si Willard está en la salida y ha rescatado a Kurtz.

2.3 Perceptos

En cada turno, `get_percepts()` devuelve un diccionario de booleanos calculado a partir de las casillas adyacentes a Willard (arriba, abajo, izquierda y derecha), resumido en la Tabla 3.

Tabla 3. Perceptos de cada parte.

Parte 1 (8 perceptos)	**Parte 2 (10 perceptos)**
Brisa: algún precipicio adyacente	`eF`, olor a queroseno: fuego adyacente
Ronquido: soldado adyacente	`eP`, crujido: pinchos adyacentes
Resplandor: salida adyacente o en la casilla actual	`eD`, cables en el suelo: dardos adyacentes
	`eM`, ronquido: soldado adyacente
	`eS`, resplandor: salida adyacente
En ambas: `Pared_U`, `Pared_D`, `Pared_L`, `Pared_R` (paredes) y `Grito` (la granada ha matado al soldado).	



Figura 1. Parte 1: cuadrícula al inicio de la partida (izquierda) y perceptos del primer turno (derecha).

3 Parte 1: agente lógico

3.1 Base de conocimiento

La base de conocimiento (KB) se almacena en la clase `GameState` y está formada por las siguientes listas:

- `visited`: casillas visitadas.
- `history`: casillas visitadas junto con los perceptos recibidos en cada una.
- `possible_P`, `possible_S`, `possible_E`: posibles ubicaciones de los precipicios, del soldado y de la salida. Empiezan con todas las casillas y se reducen a medida que se obtiene información.
- `confirmed_P`: precipicios cuya ubicación se conoce con certeza. Es necesaria porque, al haber tres precipicios, saber que una casilla lo es no permite descartar las demás.

3.2 Inferencia

La función `update_kb()` recorre el historial en cada turno y aplica las siguientes reglas de inferencia proposicional:

- Toda casilla visitada es segura: no contiene precipicios ni soldado. Mientras no se haya encontrado la salida, tampoco puede ser la salida.
- La ausencia de brisa, ronquido o resplandor descarta el elemento correspondiente en todas las casillas adyacentes.
- Un ronquido o un resplandor restringen las posibles ubicaciones del soldado o de la salida a las casillas adyacentes. Al ser elementos únicos, la información de varios turnos se combina por intersección.
- La brisa no permite descartar casillas, porque puede deberse a cualquiera de los tres precipicios. Si en la casilla actual hay brisa y solo queda una casilla adyacente que pueda ser precipicio, esa casilla se confirma como precipicio.
- Una granada fallida descarta la casilla objetivo como posición del soldado.

3.3 Flujo de ejecución e interfaz

La función `main()` ejecuta un bucle en el que cada iteración es un turno: se imprime el tablero y el estado de Willard, se calculan y muestran los perceptos, se actualiza la KB y se muestran las inferencias, y se pide al jugador la siguiente acción. El bucle termina cuando Willard muere o sale del palacio con Kurtz.

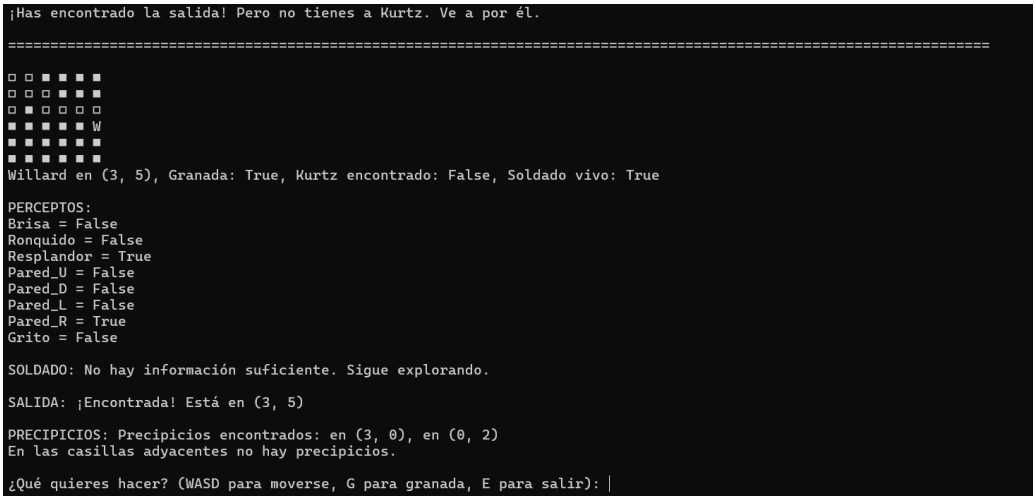


Figura 2. Parte 1: ejemplo de turno completo, con tablero, estado, perceptos, inferencias y petición de acción.

En cada turno el jugador ve un mensaje (si ha ocurrido algo, como encontrar a Kurtz o la salida), el tablero (las casillas visitadas aparecen vacías y las desconocidas, rellenas), su estado y la información inferida sobre el soldado, la salida y los precipicios. Para el soldado y la salida, mientras no se haya recibido ningún ronquido o resplandor se indica que no hay información suficiente, ya que pueden estar en cualquier casilla no explorada. A partir de la primera señal, el jugador sabe en todo momento dónde pueden estar, con la dirección si son adyacentes. Para los precipicios se indican los confirmados y las casillas adyacentes que podrían serlo.

```
SOLDADO: No hay información suficiente. Sigue explorando.

SALIDA: ¡Encontrada! Está en (3, 5)

PRECIPICIOS: Podría haber precipicios: abajo (4, 1), a la izquierda (3, 0)
Precipicios encontrados: a la izquierda (3, 0), en (0, 2)
```

Figura 3. Parte 1: indicadores de dirección para el soldado, la salida y los precipicios.

3.4 Resultados

El agente guía al jugador usando solo casillas completamente seguras. En pocos turnos se acota dónde pueden estar el soldado y la salida, y con suficiente exploración se puede llegar a conocer la ubicación de los tres precipicios.

Resulta especialmente interesante el contraste entre un peligro único y tres peligros que emiten la misma señal. Con el soldado y la salida, una señal positiva permite descartar todas las casillas no adyacentes; con los precipicios no, porque la brisa puede deberse a cualquiera de ellos, o incluso a dos a la vez. Por eso, para localizar los tres precipicios hay que explorar prácticamente todo el mapa.

4 Parte 2: agente bayesiano

4.1 Creencias iniciales

El agente bayesiano trabaja sobre el palacio de la parte 2 (Tabla 2). Para cada elemento $X \in \{F, P, D, M, S\}$ mantiene una matriz de creencias de 6×6 con la probabilidad $P(X \text{ en } (i, j))$. La distribución inicial es uniforme sobre todas las casillas salvo la de partida: $\pi_{00} = 0$ y $\pi_{ij} = 1/35$ en las demás.

```
□ ■ ■ ■ ■ ■
□ ■ ■ ■ ■ ■
□ ■ ■ □ ■ W
□ □ □ □ □ □
□ ■ ■ □ ■ ■
■ ■ ■ ■ ■ ■
Willard en (2, 5), Granada: True, Kurtz encontrado: False, Soldado vivo: True

PERCEPTOS:
eF = False
eP = False
eD = False
eM = False
eS = True
Pared_U = False
Pared_D = False
Pared_L = False
Pared_R = True
Grito = False
```

Figura 4. Parte 2: cuadrícula y perceptos de un turno.

4.2 Actualización con la regla de Bayes

En cada turno, para cada elemento X y su percepto e, se aplica la regla de Bayes en todas las casillas:

$$P(X_{ij} \mid e) \;\propto\; P(e \mid X_{ij})\, P(X_{ij}). \tag{1}$$

La verosimilitud es determinista, de acuerdo con el modelo del enunciado. Si el percepto es positivo, vale 1 en las casillas adyacentes a Willard y 0 en el resto; si es negativo, vale 0 en las adyacentes y 1 en el resto. Tras multiplicar por la creencia previa, se normaliza la matriz para

que sume 1.

Además, se fuerzan a 0 las probabilidades de todas las casillas visitadas, ya que Willard ha sobrevivido en ellas. Cuando se encuentra la salida, su distribución se concentra en su casilla real. La granada también actualiza las creencias sobre el soldado: si lo mata, su matriz pasa a ser nula; si falla, se anula la casilla objetivo.

4.3 Riesgo de cada casilla

Para decidir qué movimientos son seguros, el agente combina las creencias de las tres trampas y del soldado.

Probabilidad de trampa. Las tres trampas se colocan de forma independiente (pueden coincidir en una casilla) y cada percepto depende solo de su propia trampa. Por tanto, las creencias sobre las tres trampas siguen siendo independientes, y la probabilidad de que una casilla contenga al menos una trampa es

$$P(\text{trampa}_{ij}) \;=\; 1 - \big(1 - P(F_{ij})\big)\big(1 - P(P_{ij})\big)\big(1 - P(D_{ij})\big). \tag{2}$$

Por ejemplo, si un olor a queroseno indica que el fuego está en una de cuatro casillas, cada una tiene una probabilidad de trampa de 0,25.

Probabilidad de peligro. El soldado nunca se coloca en una casilla con trampa, así que los sucesos «hay una trampa» y «está el soldado» son incompatibles, y la probabilidad de peligro es su suma:

$$P(\text{peligro}_{ij}) \;=\; P(\text{trampa}_{ij}) + P(M_{ij}). \tag{3}$$

Un movimiento se considera seguro si la casilla de destino tiene $P(\text{peligro}) < 0{,}1$. Si ningún movimiento cumple esta condición, el agente indica cuál es el menos arriesgado y su probabilidad de peligro.

4.4 Mapas de calor

A partir de las creencias actualizadas se dibujan tres mapas de calor con `seaborn`: la probabilidad de que haya alguna trampa en cada casilla, la probabilidad del soldado y la probabilidad de la salida. Cada casilla muestra su valor numérico, y la posición de Willard se marca con un círculo azul.

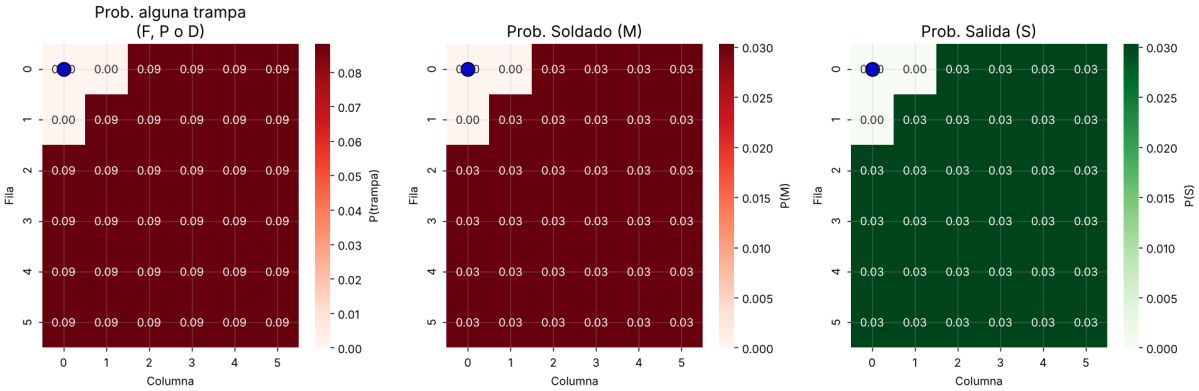


Figura 5. Mapas de calor en el primer turno de una partida. Los perceptos negativos anulan el peligro en las casillas adyacentes.

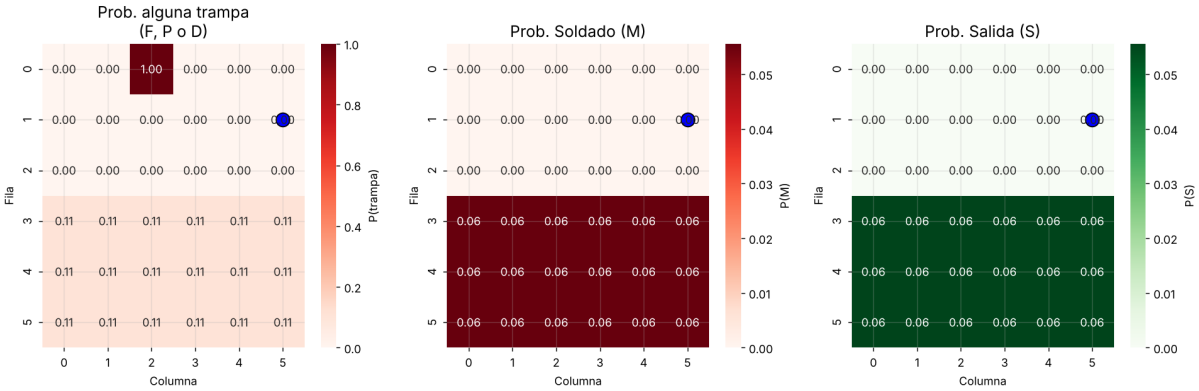


Figura 6. Mapas de calor en el turno 9 de la misma partida. El agente ha localizado con certeza la trampa de pinchos en $(0, 2)$, y las casillas que siguen sin explorar tienen una probabilidad de trampa de 0,11.

4.5 Flujo de ejecución

Cada turno sigue los siguientes pasos:

1. Se imprime la cuadrícula: solo se ven las casillas visitadas y la posición de Willard.
2. Se calculan y muestran los perceptos de la casilla actual.
3. Se actualizan las creencias con `update_beliefs()`.
4. Se dibujan los tres mapas de calor.
5. Se calculan y muestran los movimientos seguros.
6. El jugador elige su acción: moverse, lanzar la granada o intentar salir.

```
================================================================================================================

W ■ ■ ■ ■ ■
■ ■ ■ ■ ■ ■
■ ■ ■ ■ ■ ■
■ ■ ■ ■ ■ ■
■ ■ ■ ■ ■ ■
■ ■ ■ ■ ■ ■
Willard en (0, 0), Granada: True, Kurtz encontrado: False, Soldado vivo: True

PERCEPTOS:
eF = False
eP = False
eD = False
eM = False
eS = False
Pared_U = True
Pared_D = False
Pared_L = True
Pared_R = False
Grito = False

Mostrando heatmaps...

Movimientos seguros (p < 0.1): a la derecha, abajo

¿Qué quieres hacer? (WASD para moverse, G para granada, E para salir):
```

Figura 7. Parte 2: salida completa de un turno.

4.6 Resultados

La inferencia bayesiana refina las creencias de forma intuitiva. Un percepto positivo como `eM` anula la probabilidad del soldado en todo el tablero salvo en las casillas adyacentes, que pasan a repartirse la probabilidad (entre 0,25 y 0,5 cada una, según cuántas casillas adyacentes haya). Un percepto negativo anula el elemento en las casillas adyacentes y eleva ligeramente la probabilidad del resto; por ejemplo, en el primer turno pasa de $1/35 \approx 0{,}029$ a $1/33 \approx 0{,}030$.

Con un umbral de 0,1 el agente es conservador: solo recomienda casillas en las que los perceptos han descartado prácticamente todo peligro. Una casilla sin ninguna información tiene una probabilidad de peligro de entre 0,11 y 0,12 al principio de la partida (tres trampas y un soldado repartidos entre las casillas restantes), por lo que nunca se considera segura sin evidencia.

5 Parte 3: proceso de decisión de Markov

5.1 El río

El capitán y Kurtz deben cruzar un río representado por una cuadrícula de 7 filas y 6 columnas. Las columnas de los extremos son orillas sin corriente; en cada columna interior la corriente tiene una intensidad aleatoria $rs_j \in [0{,}06;\ 0{,}94]$ que empuja hacia abajo. En el río hay dos islas en posiciones aleatorias, y la salida se sitúa en una fila aleatoria de la orilla derecha. El capitán empieza en $(0, 0)$.

5.2 Componentes del MDP

- **Estados:** las casillas de la cuadrícula que no son islas.
- **Acciones:** $\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}$. La acción «quedarse» no se elige: aparece cuando un movimiento choca con un borde o con una isla.
- **Transiciones:** con una acción distinta de *down*, el capitán se mueve en la dirección elegida con probabilidad $1 - rs_j$ y la corriente lo arrastra hacia abajo con probabilidad rs_j. La acción *down* es determinista. Si un movimiento choca con un borde o con una isla, el capitán se queda en su casilla, y esa probabilidad se acumula en «quedarse».
- **Recompensas:** $R(s') = +100$ al llegar a la salida y -1 por cada paso, para minimizar su número. El código define también -100 para las islas, pero no llega a aplicarse: como un movimiento hacia una isla deja al capitán en su casilla, las islas actúan como obstáculos, y chocar con ellas solo cuesta un paso.
- **Estado terminal:** la salida. La partida acaba al alcanzarla, por lo que su valor es 0 y la recompensa de $+100$ se recibe al entrar en ella.
- **Factor de descuento:** $\gamma = 0{,}95$.

5.3 Value iteration

La política óptima se obtiene iterando la ecuación de Bellman desde $V_0(s) = 0$:

$$V_{k+1}(s) \;=\; \max_{a \in \mathcal{A}} \sum_{s'} P(s' \mid s, a)\,\big[R(s') + \gamma\, V_k(s')\big]. \tag{4}$$

Para cada estado se calcula $Q(s, a)$ para las cuatro acciones, se toma $V(s) = \max_a Q(s, a)$ y se guarda la acción que alcanza el máximo como $\pi(s)$. Los valores se actualizan sobre la misma matriz a medida que se recorren los estados, una variante que converge al mismo punto fijo. El algoritmo se detiene cuando ningún valor cambia más de $\theta = 10^{-6}$ entre dos iteraciones.

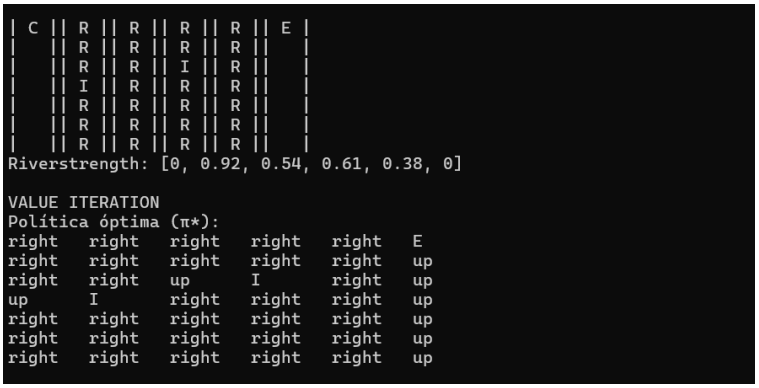


Figura 8. Ejemplo de río (C: capitán, R: río, I: isla, E: salida), intensidad de la corriente por columna y política óptima π^*.

5.4 Simulación

Con la política óptima se simulan diez partidas desde $(0, 0)$. En cada paso se aplica la acción $\pi^*(s)$ y se sortea el estado siguiente según $P(s' \mid s, a)$. Se muestra si el movimiento se debe a la corriente y la recompensa acumulada. Como cada paso cuesta 1 y la salida da 100, la recompensa de una partida de k pasos es $100 - (k - 1)$.

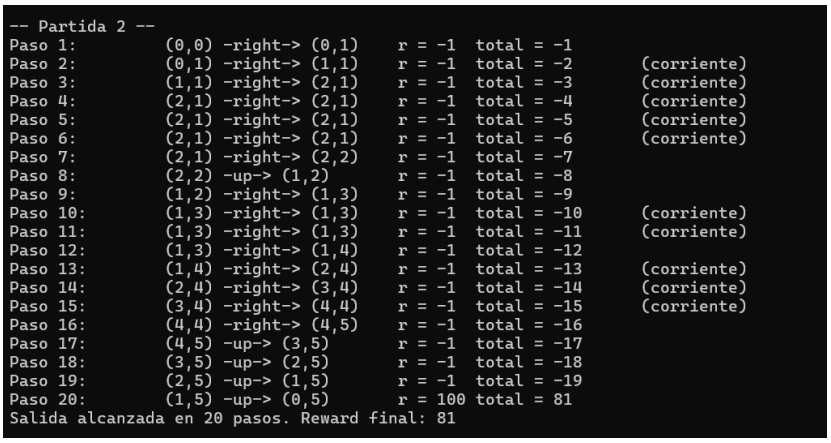


Figura 9. Simulación de una partida con la política óptima.

5.5 Resultados

Para caracterizar el comportamiento del programa se ha ejecutado sobre 200 ríos generados aleatoriamente (Tabla 4).

Tabla 4. Resultados del MDP sobre 200 ríos aleatorios.

Magnitud	Valor
Iteraciones hasta la convergencia	mediana 73 (entre 19 y 172)
Valor óptimo del estado inicial, $V^*(0, 0)$	mediana 40,5 (entre 4,6 y 64,1)
Recompensa media de 10 partidas	mediana 86,9 (el 80 % de los ríos, entre 73,5 y 91,1)
Pasos por partida	media 16,9; mediana 13

La política óptima hace lo que cabe esperar de ella: evita chocar con las islas, que solo harían perder pasos, tiende a ir hacia la derecha cuando puede y, para rodear una isla, sube por las

columnas con poca corriente y baja por las de corriente fuerte. Aprovecha las columnas en las que es menos probable que la corriente lo arrastre para ganar altura, y deja que la corriente actúe donde la caída es menos perjudicial.

6 Conclusiones

Los tres programas muestran tres formas de razonar en un mismo escenario. El agente lógico trabaja con certezas: solo afirma lo que se deduce de los perceptos, lo que lo hace fiable pero limitado cuando varios peligros producen la misma señal. El agente bayesiano cuantifica la incertidumbre y permite decidir con un nivel de riesgo controlado; para ello es esencial combinar correctamente las creencias, teniendo en cuenta qué elementos son independientes y cuáles incompatibles. Finalmente, el MDP del río introduce la toma de decisiones secuencial con transiciones estocásticas: *value iteration* encuentra una política que aprovecha o evita la corriente según la columna, y en la mitad de las partidas simuladas llega a la salida en 13 pasos o menos.